

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	K Sravan Kumar	Reg. No.:	192311210
Section / Batch:	SLOT-B	Date:	23-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Q20. Autonomous Vehicle Path Planning Optimization (Graph Algorithms)

An autonomous vehicle must compute the safest and shortest path in a dynamic environment. Analyze how graph algorithms can be used for path planning. Identify constraints such as obstacles and real-time updates. Design a solution using shortest path techniques. Discuss scalability and adaptability. Evaluate computational efficiency. Interpret results in real-world autonomous driving scenarios.

Code of Conduct

I K Sravan Kumar / 192311210 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K Sravan Kumar

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____

Q20. Autonomous Vehicle Path Planning Optimization – Graph Algorithms

1. Introduction

Autonomous vehicles must continuously select a safe and efficient route from their current position to a destination. The environment may contain moving vehicles, pedestrians, cyclists, road closures, construction zones, traffic congestion, and uncertain sensor observations. Path planning can be modeled as a graph problem in which locations or road states are represented as vertices and feasible movements are represented as weighted edges.

The edge weight can combine travel distance, estimated travel time, collision risk, road quality, and energy consumption. A shortest-path algorithm can then minimize a carefully designed cost function instead of distance alone.

Important graph algorithms include:

- Dijkstra's Algorithm – optimal for non-negative edge costs.
- A* Search – uses a heuristic to focus exploration toward the destination.
- Dynamic/incremental replanning – updates the route when obstacles or road conditions change.
- Bidirectional search – can reduce search effort when start and goal are known.

For an autonomous vehicle, the selected algorithm must satisfy both optimality and real-time constraints. A mathematically shortest route is not useful if it is unsafe or takes too long to compute.

2. Industry Problem Understanding

- Vehicle state: current position, heading, speed and lane.
- Goal state: destination or next navigation waypoint.
- Road network: lanes, intersections and legal driving transitions.
- Dynamic obstacles: vehicles, pedestrians and cyclists.
- Static obstacles: barriers, medians and restricted areas.
- Optimization objectives: safety first, followed by travel time, distance and energy.

3. Graph Representation

Let the road environment be modeled as a weighted directed graph $G = (V, E)$. Each vertex represents a navigable state such as a lane segment, intersection, grid cell, or sampled configuration. Each directed edge represents a legal movement from one state to another.

A useful cost function is:

$$\text{Cost}(e) = \alpha \cdot \text{distance}(e) + \beta \cdot \text{time}(e) + \gamma \cdot \text{risk}(e) + \delta \cdot \text{energy}(e)$$

where α , β , γ and δ are application-specific weights. Safety can also be enforced as a hard constraint by removing edges whose predicted collision risk exceeds a threshold.

Example road graph

Node	Meaning	Example outgoing edges
A	Start position	$A \rightarrow B, A \rightarrow C$
B	Lane segment	$B \rightarrow D, B \rightarrow E$
C	Alternative lane	$C \rightarrow D$
D	Intersection	$D \rightarrow E, D \rightarrow G$
E	Road segment	$E \rightarrow F, E \rightarrow G$
F	Goal approach	$F \rightarrow G$
G	Destination	—

For a grid-based local planner, each cell can be a node and neighboring cells can be connected by edges. Obstacles are represented by blocked cells or by removing unsafe edges. For a lane-level planner, nodes can represent lane positions and edges can represent permitted lane changes and turns.

4. Key Constraints

- Obstacle avoidance — static and moving obstacles must not be entered.
- Real-time updates — sensor information can invalidate the current route.
- Vehicle dynamics — steering, braking, acceleration and turning radius constrain feasible edges.
- Traffic rules — one-way roads, lane restrictions and signals must be respected.
- Safety margin — the planner should preserve a buffer around predicted obstacle positions.
- Computational budget — a route must be found within a strict time limit.

5. Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm repeatedly selects the unvisited vertex with the smallest tentative distance and relaxes its outgoing edges. With non-negative edge weights, it returns an optimal shortest path from the source to all reachable vertices.

For autonomous driving, Dijkstra can be used when the road graph is relatively stable and the cost function contains non-negative values. Its major advantage is correctness and predictable behavior. Its limitation is that it explores many states that may not lie near the destination.

Algorithm steps

- Initialize source distance to 0 and all other distances to infinity.
- Insert the source into a min-priority queue.
- Extract the node with the smallest current distance.
- Relax each valid outgoing edge.
- Update a node when a lower-cost route is discovered.
- Stop when the destination is reached or the queue is empty.
- Reconstruct the route using predecessor information.

Time complexity: with an adjacency list and binary heap, approximately $O((V + E) \log V)$, commonly written $O(E \log V)$ for connected sparse graphs. Memory usage is $O(V + E)$.

6. A* Search for Faster Point-to-Point Planning

A* combines the path cost already traveled, $g(n)$, with a heuristic estimate $h(n)$ from node n to the goal: $f(n) = g(n) + h(n)$. On a road grid, Euclidean or Manhattan distance can provide a useful heuristic when it does not overestimate the remaining cost.

An admissible heuristic preserves shortest-path optimality. Because A* prioritizes states that appear promising toward the destination, it can inspect fewer states than uninformed Dijkstra on many spatial maps.

7. Proposed Path-Planning Architecture

The proposed solution uses a layered architecture so perception, prediction, graph construction, route search and vehicle control can be updated independently.

Module	Input	Output	Main responsibility
1. Perception	Camera/LiDAR/radar/GNSS	Detected objects and free space	Identify road and obstacle information
2. Prediction	Tracked object states	Predicted trajectories	Estimate future obstacle positions
3. Graph Builder	Map + free space + predictions	Weighted graph	Create feasible nodes and edges
4. Path Planner	Graph + start + goal	Optimal/safe route	Run A*/Dijkstra
5. Replanner	New sensor/map events	Updated route	Repair or recompute path
6. Controller	Planned trajectory	Steering/braking commands	Track path safely

Processing flow

Sensors → Perception → Prediction → Graph Update → A* Search → Safety Check → Trajectory → Vehicle Controller

The graph is not assumed to remain constant. New obstacles or road changes trigger an update. A local planner can operate on a smaller rolling map around the vehicle while a global planner supplies a longer route.

8. Dynamic Environment Handling

- Mark affected nodes or edges as blocked or increase their risk cost.
- If the current route remains safe, continue following it to avoid unnecessary replanning.
- If the route becomes invalid, run a local A* search from the current state to a safe waypoint.
- Use time-stamped obstacle predictions so the planner reasons about future conflicts.
- Apply route-change penalties to prevent unstable switching between nearly equal paths.

9. Sample Weighted Scenario

Consider a simplified local road graph. Edge weights represent a combined safety and travel cost; lower values are preferred.

Edge	Distance cost	Risk cost	Total cost
A→B	4	1	5
A→C	2	4	6
B→D	3	1	4
C→D	2	1	3
B→E	4	2	6
D→E	2	1	3
D→G	6	2	8
E→G	3	1	4

One candidate route is A→C→D→E→G with total cost $6 + 3 + 3 + 4 = 16$. Another is A→B→D→E→G with cost $5 + 4 + 3 + 4 = 16$. The planner therefore needs a tie-breaking policy, such as greater safety margin, fewer lane changes, or lower travel time.

10. Obstacle Update Example

Suppose a moving vehicle is predicted to occupy edge D→E during the next planning interval. The planner can temporarily block that edge or assign it a very high risk cost. The revised search then chooses another feasible route, such as D→G if its cost is acceptable, or waits/replans if no safe route exists.

The planner should never simply choose a mathematically shorter path through an unsafe region. Safety constraints are applied before optimization.

11. A* Algorithm – Pseudocode

```
A_STAR(graph, start, goal):
    open_set = priority queue ordered by f = g + h g[start]
    = 0
    parent[start] = NONE
    push start with priority h(start)

    while open_set is not empty:
        current = pop node with smallest f

        if current == goal:
            return reconstruct_path(parent, goal)

        for each edge (current, next) in graph: if
            edge is blocked:
                continue

            tentative = g[current] + cost(edge)

            if next not in g OR tentative < g[next]: g[next]
                = tentative
                parent[next] = current
                f = tentative + heuristic(next, goal)
                push/update next with priority f

    return NO_SAFE_PATH
```

12. C++ Implementation Example

```
#include <bits/stdc++.h> using
namespace std;

struct Node {
    int id;
    double f;
    bool operator>(const Node& other) const { return f > other.f; }
};

double heuristic(int a, int goal,
    const vector<pair<double,double>>& pos) { double
    dx = pos[a].first - pos[goal].first;
    double dy = pos[a].second - pos[goal].second; return
    sqrt(dx*dx + dy*dy);
}

vector<int> astar(int start, int goal,
    const vector<vector<pair<int,double>>>& adj, const
    vector<pair<double,double>>& pos) {
    int n = adj.size();
    const double INF = 1e18;
    vector<double> g(n, INF);
    vector<int> parent(n, -1);
    priority_queue<Node, vector<Node>, greater<Node>> pq;

    g[start] = 0;
    pq.push({start, heuristic(start, goal, pos)});

    while (!pq.empty()) {
        int u = pq.top().id;
        pq.pop();
        if (u == goal) break;

        for (auto [v, w] : adj[u]) {
            double ng = g[u] + w; if
            (ng < g[v]) {
                g[v] = ng;
                parent[v] = u;
                pq.push({v, ng + heuristic(v, goal, pos)});
            }
        }
    }

    vector<int> path;
    if (g[goal] == INF) return path;
    for (int v = goal; v != -1; v = parent[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
```



```
    return path;  
}
```

13. Safety-Aware Cost Design

Distance-only optimization is inadequate for autonomous vehicles. A safe planner should incorporate multiple factors into the edge cost while also enforcing hard constraints.

Factor	Example representation	Planning effect
Distance	Meters	Prefers shorter routes
Travel time	Seconds	Avoids slow road segments
Collision risk	Predicted risk score	Penalizes unsafe segments
Clearance	Distance from obstacles	Prefers larger safety margins
Lane changes	Count or maneuver cost	Reduces unnecessary maneuvers
Energy	Estimated battery/fuel cost	Supports efficient driving
Comfort	Curvature/acceleration penalty	Reduces harsh motion

$$J = \alpha D + \beta T + \gamma R + \delta L + \epsilon E$$

The weights must be calibrated and tested; they should not be chosen only to make one example produce a desired route. Hard constraints include collision avoidance, road legality, maximum steering rate, vehicle dimensions, stopping distance and minimum clearance.

14. Real-Time Replanning Strategy

- Global planner: computes a route over the broader road network.
- Local planner: searches a smaller region at high frequency.
- Event trigger: obstacle detection, road closure, route deviation or map update.
- Incremental update: change only affected edges where possible.
- Validation: check collision risk, legal movement and vehicle feasibility before execution.

15. Dynamic Replanning Module

```
REPLAN_LOOP:
  sense environment
  detect and track obstacles
  predict obstacle motion

  update local graph: block
    unsafe edges update
    risk/time costs
    add newly available edges

  if current path is invalid OR estimated
    risk > threshold OR route cost
    increases significantly:
      run A* from current state to target waypoint

  verify new path against safety constraints send
  feasible trajectory to controller repeat at the
  next planning cycle
```

16. Scalability

A city-scale road network may contain a very large number of vertices and edges. Running an exhaustive search over the entire graph at every sensor update is inefficient. Scalability can be improved by dividing planning into levels.

- Hierarchical planning: global route on road graph, local route on nearby lane/grid states.
- Spatial partitioning: search only regions near the vehicle and route corridor.
- Priority queues: efficiently select promising states.
- Heuristics: A* reduces exploration when the heuristic is informative.
- Caching: reuse portions of previous routes when the environment has not changed.
- Parallel processing: perception, prediction and planning can run as separate tasks.

17. Adaptability

The same graph framework can adapt to different vehicle types and driving conditions by changing the graph, constraints and edge-cost model. A delivery vehicle may prioritize energy and legal access, while an emergency vehicle may use different road-priority rules.

18. Alternative Algorithms and Their Roles

Algorithm	Strength	Limitation	Use in autonomous driving
Dijkstra	Optimal for non-negative costs	Explores many states	Baseline/global shortest path
A*	Goal-directed and usually faster	Needs a good heuristic	Point-to-point planning
Bidirectional Search	Searches from both ends	More complex with dynamic constraints	Static route queries
D* / incremental replanning	Efficient route repair after changes	More complex implementation	Dynamic environments
Greedy Best-First	Very fast in some cases	Not guaranteed optimal	Fast approximate fallback

19. Recommended Approach

A* for local point-to-point path planning + dynamic graph updates + safety validation is recommended. Dijkstra can be retained as a baseline for correctness and performance comparison. For highly dynamic road networks, incremental replanning techniques can reduce repeated work.

The architecture does not depend on a single algorithm. The planner can switch strategy according to map size, obstacle frequency, available computation and required response time.

20. Suitable Data Structures

- Adjacency list for sparse road graphs.
- Min-priority queue / binary heap for A* and Dijkstra.
- Hash/set structures for blocked nodes and edges.
- Parent array or map for route reconstruction.
- Spatial index/grid for locating nearby obstacles and graph states.
- Timestamped obstacle records for dynamic validity checks.

21. Tool Support and Experimental Setup

Tool	Purpose
C++	Efficient implementation of A*, Dijkstra and graph structures
Visual Studio Code	Code development and debugging
GCC/G++	Compilation and execution
Python	Performance analysis and plotting
NetworkX	Graph construction and experimentation
Matplotlib	Visualization of paths, costs and runtime
Git/GitHub	Version control and experiment tracking

22. Experimental Method

- Create road graphs of increasing size and density.
- Generate static and dynamic obstacle scenarios.
- Run Dijkstra and A* from the same start and goal states.
- Record path cost, path length, nodes expanded, runtime and memory.
- Repeat experiments under different obstacle densities and update frequencies.
- Validate that returned routes do not contain blocked edges or violate hard constraints.

The evaluation should include both normal and stress cases. A planner that is fast on an empty map but fails when obstacle updates become frequent is not suitable for a real autonomous vehicle.

23. Validation Criteria

- Path reaches the destination when a feasible route exists.
- No selected edge intersects a prohibited obstacle region.
- Route obeys direction, lane and maneuver constraints.
- Reported cost matches the sum of selected edge costs.
- Replanning produces a safe alternative after route blockage.
- Runtime remains within the application's planning-cycle budget.

24. Performance Evaluation

The computational efficiency of the planner should be measured using multiple metrics.

Metric	Interpretation
Planning time	Time required to produce or repair a route
Nodes expanded	Amount of graph explored
Edges evaluated	Search effort
Path cost	Quality of selected route
Path length	Physical travel distance
Memory usage	Scalability of graph and search structures
Replanning time	Responsiveness after changes
Safety violations	Must remain zero for accepted routes

Expected qualitative result

- Dijkstra and A* should return the same optimal path when A* uses an admissible heuristic.
- A* is expected to expand fewer nodes in many spatial road graphs.
- As obstacle density increases, replanning events may increase.
- As graph size grows, hierarchical/local planning becomes increasingly important.
- Safety constraints may intentionally produce a slightly longer route than pure distance minimization.

These are expected trends, not claimed benchmark measurements. Actual runtime and node counts should be obtained from the implemented program and selected hardware.

25. Complexity Analysis

- Dijkstra with a binary heap: $O((V + E) \log V)$.
- A* with a binary heap: worst-case comparable to Dijkstra; practical work depends on the heuristic.
- Adjacency-list graph storage: $O(V + E)$.
- Path reconstruction: $O(\text{number of edges in the returned path})$.
- Localized graph updates improve scalability compared with full graph reconstruction after every sensor update.

26. Real-World Autonomous Driving Interpretation

In a real driving scenario, the vehicle may be traveling on a multi-lane urban road when a pedestrian or stalled vehicle appears ahead. The perception system detects the object and the prediction module estimates its future location. The graph builder marks the affected lane segment as unsafe or increases its risk cost.

If the current path is no longer valid, the local A* planner searches neighboring lanes and road segments. It may choose a safe lane change followed by a slightly longer route. The controller converts the selected route into a smooth trajectory while respecting vehicle dynamics.

Graph search provides the discrete decision layer, while vehicle control provides continuous motion execution. A path that is valid on a graph must still be checked for physical feasibility before commands are issued.

27. Scenario-Based Analysis

Scenario	Planner response	Expected outcome
Clear road	Normal A* search	Shortest/lowest-cost route
Static roadblock	Block affected edges	Safe detour
Slow traffic	Increase time cost	Alternative faster route if worthwhile
Moving obstacle	Use predicted occupancy	Avoid future collision
Temporary lane closure	Update local graph	Replan around closure
No safe route	Reject unsafe paths	Controlled fallback / safe stop

28. Practical Safety Principle

Safety should be treated as a constraint, not merely as a small numerical preference. If an edge is predicted to cause a collision or violate a mandatory driving rule, it should be removed from the feasible graph regardless of how short it is.

29. Key Issues Identified

- Dynamic obstacles can invalidate a previously optimal route.
- Sensor uncertainty means obstacle positions and predictions are not exact.
- A distance-only objective can select an unsafe or uncomfortable route.
- Large maps increase graph-search and memory requirements.
- Frequent updates can cause excessive replanning if not controlled.
- Vehicle kinematics restrict which graph transitions are actually drivable.
- Traffic rules and lane semantics must be encoded in the graph.
- A safe fallback is required when no feasible route is available.

30. Advantages of Graph-Based Path Planning

- Provides a clear mathematical representation of navigable states and transitions.
- Supports well-studied shortest-path algorithms with predictable behavior.
- Can combine distance, time, risk and energy in weighted edges.
- A* can exploit spatial heuristics for faster point-to-point search.
- Dynamic updates can block or reweight affected parts of the graph.
- Hierarchical graphs support large road networks.
- Results can be validated using explicit path and safety constraints.

31. Limitations

- Graph discretization may lose fine details of continuous vehicle motion.
- A poor heuristic or graph design can reduce A* efficiency.
- Accurate dynamic obstacle prediction is difficult in crowded environments.
- Edge weights depend on estimates that may change quickly.
- Frequent full graph reconstruction may be computationally expensive.
- Shortest-path optimality alone does not guarantee smooth or comfortable vehicle motion.

32. Scalability and Adaptability Analysis

For small local maps, standard A* can be sufficient. As the environment grows, the system should avoid repeatedly searching every road segment. A hierarchical approach can divide the problem into global route selection and local maneuver planning.

Scale	Recommended strategy	Reason
Small local grid	A*	Simple and fast point-to-point search
Large static road map	Hierarchical A* / indexed graph	Reduces search region
Frequent obstacle changes	Incremental/dynamic replanning	Reuses previous planning work
Dense urban traffic	Local prediction + risk-aware graph	Handles dynamic interactions
Resource-constrained vehicle	leBounded local search	Controls CPU and memory use

Adaptability is achieved by updating the graph rather than redesigning the entire algorithm for every situation. New road closures, temporary obstacles, changed speed limits and traffic estimates can be represented by modifying edge availability or weights.

33. Optimization Opportunities

- Use a consistent and admissible heuristic for A*.
- Limit local search to a corridor around the current/global route.
- Reuse unaffected portions of the previous route.
- Precompute static road-network information where possible.
- Use efficient adjacency lists and priority queues.
- Separate safety filtering from cost optimization so unsafe edges are rejected early.

34. Recommended End-to-End Workflow

Map and sensor data → obstacle detection → obstacle prediction → graph update → safety filtering → A* search → path validation → trajectory generation → vehicle control → continuous monitoring and replanning.

35. Results and Discussion

The proposed graph-based solution provides a systematic way to optimize autonomous vehicle routes in both static and changing environments. Dijkstra provides a correctness-oriented baseline, while A* improves point-to-point efficiency by directing the search toward the goal.

In the sample weighted scenario, two routes have the same combined cost of 16. This demonstrates that the planner may need a secondary criterion when multiple optimal routes exist. A safety margin, lower number of lane changes, smoother curvature, or lower predicted travel time can be used as a tie-breaker.

When an obstacle blocks an edge, the graph is updated and the search is repeated or repaired. The important performance measure is not only the quality of the final path but also the time required to react to the change.

36. Expected Evaluation Table

Test	Dijkstra	A*	Interpretation
Static map	Optimal	Optimal with admissible heuristic	Compare nodes expanded and runtime
One blocked edge	Recompute	Replan locally	A* can focus on goal region
Many obstacles	More exploration	Heuristic-guided exploration	Local/hierarchical planning becomes important
Large graph	Higher search cost	Usually lower practical search effort	Measure scalability
Frequent updates	Repeated searches	Use local/incremental replanning	Measure response time

The table describes expected algorithmic behavior and should be supplemented with measured runtime and node-count results when the program is executed on a specific hardware and dataset.

37. Overall Efficiency

- Correctness: shortest feasible path under the chosen cost model.
- Responsiveness: route generation and replanning within the required cycle time.
- Scalability: manageable search effort as graph size grows.
- Adaptability: safe response to changes in obstacles and road conditions.
- Safety: no accepted path violates mandatory safety constraints.

38. Conclusion

Autonomous vehicle path planning can be effectively modeled as a weighted graph optimization problem. Vertices represent navigable vehicle states and edges represent legal, physically feasible movements. By assigning edge costs for distance, time, risk, energy and maneuver effort,

shortest-path techniques can optimize a route according to real driving objectives.

Dijkstra's algorithm provides a reliable baseline for non-negative edge costs, while A* is a strong choice for point-to-point planning because its heuristic focuses the search toward the destination. In dynamic environments, the graph must be updated when obstacles or road conditions change, and local or incremental replanning can improve responsiveness.

The proposed architecture separates perception, prediction, graph updating, path planning, replanning and vehicle control. This improves scalability and adaptability and allows the system to use different planning strategies at different levels. Real-world deployment must additionally validate vehicle dynamics, uncertainty, traffic rules, safety margins and computation time.

Therefore, a safety-constrained, risk-aware A* planner with dynamic graph updates provides an efficient and practical foundation for autonomous vehicle path planning, while performance testing on realistic maps and obstacle scenarios is essential before deployment.